CYBR 322 Computer Security and Fundamentals

Cryptographic Tools

(10 pts) Name: Student ID:

After you finish all tasks, you extract the file into PDF and sign it with your certificated signature.

(10 pts) What is the difference between symmetric and asymmetric encyption?

(10 pts) List three approaches to message authentication.

(10 pts) What properties must a hash function have to be useful for message authentication?

(30 pts) Encryption is the process of turning readable text (plaintext) into unreadable text (ciphertext) so that only someone with the right key can read it again. In this assignment, you will implement a simple encryption method using the **XOR (exclusive OR)** operation.

XOR is a logical operation that works on numbers at the binary level. One special property makes it useful for encryption:

(A XOR K) XOR K = A

This means if you encrypt something with XOR and the same key, you can decrypt it by applying XOR again with the same key. That is the essence of **symmetric encryption**: the same key is used for both encryption and decryption.

Your Task

1. Write a function that encrypts a message using XOR and a numeric key.
2. Write another function that decrypts the message using the same key.
3. Demonstrate that your code works by encrypting and then decrypting the message

Starter Code:

```
def xor_encrypt(message, key):
    encrypted = ""
```

```
    for char in message:
        encrypted_char = chr(ord(char) ^ key)  # XOR character with key
        encrypted += encrypted_char
    return encrypted

def xor_decrypt(ciphertext, key):
    decrypted = ""
    for char in ciphertext:
        decrypted_char = chr(ord(char) ^ key)  # XOR again with same key
        decrypted += decrypted_char
    return decrypted



# Test your functions here
key = 23
message = "HELLO"

cipher = xor_encrypt(message, key)
print("Encrypted:", cipher)

original = xor_decrypt(cipher, key)
print("Decrypted:", original)
```

What You Should Do

1. Run the starter code and observe the results.

2. Change the value of the key and see how the ciphertext changes.
3. Try encrypting a longer message, such as "CRYPTOGRAPHY IS FUN".
4. Experiment by using the wrong key for decryption. What happens?
5. (Optional challenge) Read text from a file, encrypt it, save it, and then decrypt it back.

(30 pts) In symmetric encryption, the same secret key is used for both encryption and decryption. But this creates a problem: how do two people share the key safely in the first place?

Asymmetric encryption solves this by using **two different keys**:

- A **public key** that anyone can use to encrypt a message.
- A **private key** that only the receiver knows, which is used to decrypt.

This way, people can send you encrypted messages without ever knowing your private key. Real-world systems like RSA use very large prime numbers and advanced math, but in this assignment, we will use a **toy version** with very small numbers to make the idea clear.

Your Task

1. Implement an encryption function that uses a public key to encrypt a message.
2. Implement a decryption function that uses a private key to decrypt the ciphertext.
3. Show that encryption with the public key and decryption with the private key recovers the original message.
4. Compare this method to the XOR (symmetric) assignment.

Starter Code:

```
# Toy keys (not secure, but good for learning!)

def letter_to_num(letter):

    return ord(letter) - ord('A') + 1  # A=1, B=2, ...
```

```
def num_to_letter(num):
    return chr(num + ord('A') - 1)

public_key = 3
private_key = 7
modulus = 33

def encrypt(message, pub_key, mod):
    encrypted = []
    for char in message:
        num = letter_to_num(char)
        cipher_val = pow(num, pub_key, mod)
        encrypted.append(cipher_val)
    return encrypted

def decrypt(cipher_list, priv_key, mod):
    decrypted = ""
    for num in cipher_list:
        plain_val = pow(num, priv_key, mod)
        decrypted += num_to_letter(plain_val)
    return decrypted

# Example
```

```
message = "HI"
cipher = encrypt(message, public_key, modulus)
print("Encrypted:", cipher)

original = decrypt(cipher, private_key, modulus)
print("Decrypted:", original)
```

What You Should Do

1. Run the code and observe the ciphertext (note: it will be a list of numbers).
2. Change the message and try encrypting different words.
3. Try using the wrong key to decrypt — what happens?
4. Compare this with XOR encryption:
    - In XOR, the same key is used both ways.
    - In this toy asymmetric system, one key encrypts and a different one decrypts.